

Giving Instructions to Computers

How Computers Understand Instructions

- Computers understand instructions written in **high-level languages** like **C++**, **Java**, and **Python**.
- These languages are similar to **English**, making them easier for humans to understand and write.
- Tools like **ChatGPT** can generate code from natural language descriptions.
- A **compiler** translates the human-readable code into a language the computer can execute.

The Problem-Solving Approach

Before writing any code, follow these steps to solve a problem:

1. **Understand the Problem** : Clearly define what needs to be solved (e.g., "make tea").
2. **Identify Given Values (Inputs)** : Determine what resources or information are available (e.g., "tea leaves, milk, sugar, water").
3. **Devise an Approach** : Plan the steps to solve the problem (e.g., "first water, then tea leaves, then sugar, then milk"). There can be multiple approaches.
4. **Implement** : Execute the planned steps (e.g., "start making tea").
5. **Debug** : Identify and fix any errors or issues that arise during implementation (e.g., "why did the tea taste bad?").
6. **Optimize** : Find ways to improve the solution for better efficiency or quality (e.g., "how can I make the tea even better?").

Understanding Computer Instructions: The Coffee Machine Analogy

- Imagine a **coffee machine** : you put in powder and water, press a button, and it makes coffee.
- The machine was **taught once** how to make coffee (e.g., "take 10g powder, 200ml water, 20g sugar, heat, then serve").
- Once instructed, it can repeat the task anytime with a simple command (like pressing a button).
- Similarly, when we write code, we **teach the computer once** how to perform a task, and it can then execute it repeatedly.

Introduction to Flowcharts

- A **flowchart** is a **diagrammatic representation** of a program's logic.
- It helps visualize the sequence of steps and decisions in a program.
- Flowcharts are useful for understanding, designing, and **debugging** program logic.

Flowchart Symbols

1. **Start/End (Oval)** : Indicates the **beginning** or **end** of a program.
2. **Input/Output (Parallelogram)** : Represents operations where the program **reads data** (input) or **displays results** (output).
 - *Example Input:* Reading numbers from a user.
 - *Example Output:* Printing a calculated answer.
3. **Process (Rectangle)** : Represents any **calculation** or **assignment** of values.
 - *Example:* `sum = a + b`, `answer = number \* number \* number`.
4. **Decision (Diamond)** : Represents a point where a **question is asked**, and the program takes different paths based on a **Yes/No** or **True/False** answer.
 - *Example:* `Is number > 0?`
5. **Arrows (Flow Lines)** : Connect the symbols and indicate the **direction of flow** of the program.

Flowchart Examples and Pseudocode

1. Sum of Two Numbers

- **Problem** : Add two numbers, **A** and **B**, and display their sum.

- **Flowchart Steps** :

1. **Start**
2. **Read** A, B (Input)
3. **Process** : `Answer = A + B`
4. **Print** Answer (Output)
5. **End**

- **Pseudocode** :

...

START

READ A, B

Answer = A + B

PRINT Answer

END

...

2. Average of Two Numbers

- **Problem** : Calculate the average of two numbers, **A** and **B**.

- **Flowchart Steps** :

1. **Start**
2. **Read** A, B
3. **Process** : `Answer = (A + B) / 2`
4. **Print** Answer

5. **End**

3. Cube of a Number

- **Problem** : Calculate the cube of a given number, **N** .

- **Flowchart Steps** :

1. **Start**

2. **Read** N

3. **Process** : `Answer = N \* N \* N`

4. **Print** Answer

5. **End**

4. Decision Making: Package Offer

- **Problem** : Accept or reject a job offer based on whether the package is **greater than or equal to 10 Lakhs** .

- **Flowchart Steps** :

1. **Start**

2. **Read** Package

3. **Decision** : `Is Package < 10 Lakhs?`

- **Yes** : **Print** "Rejected"

- **No** : **Print** "Accepted"

4. **End** (from both Print paths)

5. Decision Making: Rain Check

- **Problem** : Decide whether to go for a walk based on if it's raining.

- **Flowchart Steps** :

1. **Start**

2. **Read** Weather (e.g., "Is it raining?")

3. **Decision** : `Is it raining?`

- **Yes** : **Print** "Not going for a walk"

- **No** : **Print** "Going for a walk"

4. **End**

6. Decision Making: Even or Odd Number

- **Problem** : Determine if a given number is **even** or **odd** .

- **Concept** : An even number has a remainder of **0** when divided by **2** . An odd number has a remainder of **1** .

- **Modulus Operator (%)** : In programming, the **modulus operator (%)** gives the **remainder** of a division.

- *Example:* `5 % 2` equals **1** . `4 % 2` equals **0** .

- **Single Equal (==) vs. Double Equal (===)** :

- **= (Assignment Operator)** : Assigns a value to a variable (e.g., `x = 5` means `x` now holds the value

`5`).

- **`==` (Comparison Operator)** : Checks if two values are **equal** (e.g., `x == 5`` asks "Is the value of `x`` equal to `5``?").

- **Flowchart Steps :**

1. **Start**
2. **Read** Number
3. **Decision** : ``Is Number % 2 == 0``
 - **Yes** : **Print** "Even"
 - **No** : **Print** "Odd"
4. **End**

7. Decision Making: Comparing Two Numbers (A vs. B)

- **Problem** : Compare two numbers, **A** and **B** , and state which is larger or if they are equal.
- **Flowchart Steps :**
 1. **Start**
 2. **Read** A, B
 3. **Decision** : ``Is A > B``
 - **Yes** : **Print** "A is greater"
 - **No** : Go to next decision.
 4. **Decision** : ``Is B > A`` (This path is taken if A was not greater than B)
 - **Yes** : **Print** "B is greater"
 - **No** : **Print** "A and B are equal" (This path is taken if A was not greater than B, and B was not greater than A)
 5. **End**

8. Decision Making: Positive or Negative Number

- **Problem** : Determine if a given number is **positive** , **negative** , or **zero** .
- **Concept** :
 - Numbers **greater than 0** are positive.
 - Numbers **less than 0** are negative.
 - The number **0** is neither positive nor negative.
- **Flowchart Steps :**
 1. **Start**
 2. **Read** Number
 3. **Decision** : ``Is Number > 0``
 - **Yes** : **Print** "Positive"
 - **No** : Go to next decision.
 4. **Decision** : ``Is Number < 0`` (This path is taken if Number was not greater than 0)
 - **Yes** : **Print** "Negative"

Study Notes

- **No : Print** "Zero" (This path is taken if Number was not greater than 0, and not less than 0)

5. **End**

9. Loops: Printing a Number N Times

- **Concept of Loops** : A **loop** is used to repeat a set of instructions multiple times.

- **Problem** : Print a given **Number** a specified **N** number of times.

- **Flowchart Steps** :

1. **Start**

2. **Read** Number, N (how many times to print)

3. **Process** : `Count = 1` (Initialize a counter)

4. **Decision** : `Is Count <= N?`

- **Yes** :

1. **Print** Number

2. **Process** : `Count = Count + 1` (Increment the counter)

3. Go back to the **Decision** step (`Is Count <= N?`)

- **No** : Continue to next step.

5. **End**

10. Loops: Printing N Natural Numbers

- **Problem** : Print the first **N natural numbers** (1, 2, 3, ... up to N).

- **Flowchart Steps** :

1. **Start**

2. **Read** N

3. **Process** : `Count = 1`

4. **Decision** : `Is Count <= N?`

- **Yes** :

1. **Print** Count

2. **Process** : `Count = Count + 1`

3. Go back to the **Decision** step.

- **No** : Continue to next step.

5. **End**

11. Loops: Sum of N Natural Numbers

- **Problem** : Calculate the **sum of the first N natural numbers** ($1 + 2 + 3 + \dots + N$).

- **Flowchart Steps** :

1. **Start**

2. **Read** N

3. **Process** : `Count = 1`, `Sum = 0` (Initialize counter and sum)

4. **Decision** : `Is Count <= N?`

- **Yes** :

Study Notes

1. **Process** : ``Sum = Sum + Count`` (Add current count to sum)
2. **Process** : ``Count = Count + 1``
3. Go back to the **Decision** step.
 - **No** : Continue to next step.
5. **Print** Sum
6. **End**

12. Prime Number Check

- **Problem** : Determine if a given number **N** is a **prime number** .
- **Definition of Prime Number** : A prime number is a natural number greater than **1** that has no positive divisors other than **1** and itself.
- **Logic** : To check if N is prime, we test if it's divisible by any number from **2 up to N-1** . If it's divisible by any of these, it's not prime.
- **Flowchart Steps** :
 1. **Start**
 2. **Read** N
 3. **Decision** : ``Is N <= 1?`` (Numbers less than or equal to 1 are not prime)
 - **Yes** : **Print** "Not Prime"
 - **No** : Continue to next step.
 4. **Process** : ``Divisor = 2`` (Start checking divisibility from 2)
 5. **Decision** : ``Is Divisor < N?`` (Loop from 2 up to N-1)
 - **Yes** :
 1. **Decision** : ``Is N % Divisor == 0?`` (Check if N is divisible by Divisor)
 - **Yes** : **Print** "Not Prime", then **End** (No need to check further)
 - **No** : **Process** : ``Divisor = Divisor + 1`` (Try the next divisor), then go back to ``Is Divisor < N?``
 - **No** : (If the loop finishes without finding any divisors) **Print** "Prime"
 6. **End**

Importance of Flowcharts and Pseudocode

- Flowcharts and pseudocode are excellent tools for **understanding** and **designing** program logic.
- They help in **debugging** by allowing you to trace the program's flow with sample values.
- While useful for beginners, experienced programmers often move directly to coding after mentally outlining the logic, especially for simpler problems.

Key Takeaways

- Computers follow **precise instructions** written in programming languages.
- A structured **problem-solving approach** is crucial before coding.

Study Notes

- **Flowcharts** visually represent program logic, making it easier to understand and debug.
- Key flowchart symbols include **Start/End**, **Input/Output**, **Process**, and **Decision**.
- **Loops** allow for repetitive tasks, and **decision structures** enable programs to make choices.
- Understanding concepts like the **modulus operator (%)** and the difference between **assignment (=)** and **comparison (==)** is fundamental.

Homework

- Write the **pseudocode** for the "Printing N Natural Numbers" problem.